



Maize Yield Prediction

A Beginner's Tutorial to the Project

Crop Yield Prediction Based on Weather and Soil Data — Federal University of Technology, Minna

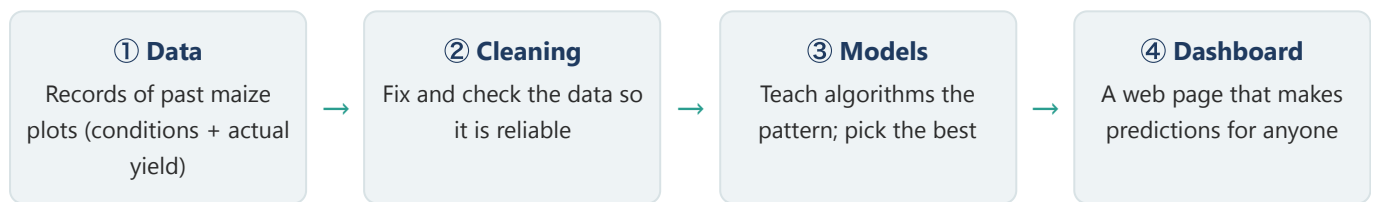
? What is this project?

This project predicts **how much maize a farm plot will produce** (its yield, in tonnes per hectare) from information about the plot — the weather it gets, its soil, and how it is managed. Instead of guessing from experience, a farmer or extension officer enters a few values and the system gives a data-driven estimate, through a simple web page anyone can use.

Under the hood it uses **machine learning**: the computer studied thousands of past maize plots, learned the pattern linking conditions to yield, and now applies that pattern to new plots.

1 The big picture

The project is a pipeline — data flows through four stages, each building on the last:



The rest of this tutorial walks through each stage in plain language, then shows you how to run and host the project yourself.

2 The data stage ①

The starting point is a spreadsheet, `crop_yield_dataset.csv`, with **10,000 records** covering four crops. Because the study is about maize, we keep only the maize rows — **2,477 plots**. Each plot lists:

- **Weather:** rainfall, temperature, humidity
- **Soil:** pH (acidity), soil type
- **Management:** fertiliser used, pesticides, planting density, irrigation method, previous crop, region
- **The answer:** the actual yield of that plot — what the model learns to predict

3 Cleaning the data stage ②

Before a model can learn, the data must be trustworthy. We removed duplicates, checked every value was physically possible (for example, soil pH between 0 and 14), and handled missing values.

The clever bit. The word "**None**" appears in the data — for example an irrigation of "None" means the plot is *rainfed*. The software we use normally treats "None" as *missing data* and would try to fill it in, which would corrupt real information. We caught this and kept "None" as a genuine category. After this fix, the maize data had **zero missing values**.

Finally, the text columns (like soil type) were turned into numbers the models can read, and the data was split into a **training set** (80%, used to learn) and a **test set** (20%, kept aside to fairly grade the models on plots they have never seen).

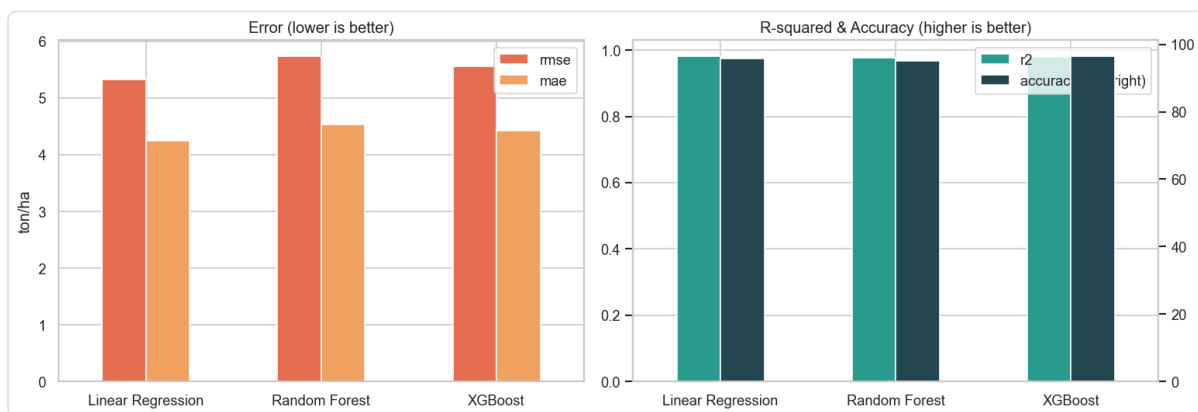
4 Training the models stage ③

We taught three different algorithms and compared them, so we could choose the best rather than trust just one:

- **Linear Regression** — the simple *baseline*. It draws the best straight-line relationship between the inputs and yield.
- **Random Forest** — builds many decision "trees" and averages them for a more robust answer.
- **XGBoost** — builds trees one after another, each one fixing the mistakes of the last.

Each model was graded on the test set. The key scores (see the glossary at the end):

Model	RMSE (lower better)	R ² (higher better)	Accuracy
Linear Regression (best)	5.325	0.981	95.77%
XGBoost	5.558	0.979	96.37%
Random Forest	5.732	0.977	95.16%



All three models compared. They perform almost identically.

The main finding. All three models explain about **98%** of the variation in yield — and the simple Linear Regression is marginally the best. Why? Because in this data, yield depends mostly on **fertiliser** and **rainfall** in a nearly straight-line way, so the simple model captures it fully. This is exactly why we included a baseline: it proves the fancier models aren't missing anything hidden.

5 The dashboard stage ④

A model is only useful if people can use it. The dashboard is a web page (built with a tool called **Streamlit**) with three parts:

- **Predict Yield** — type in a plot's conditions, choose a model, and get the predicted yield with a range and a chart of what influenced it. You can download the result as a PDF.
- **Model Performance** — compares the three models' scores.

- **Prediction History** — every prediction is saved and can be reviewed later.

It even checks your input — impossible values are refused, and unusual ones get a gentle warning.

The dashboard's prediction page — the input form and model selector.

6 How to run it on your computer

Open the **Terminal** app and type these, one at a time. First, go to the project folder:

```
cd "/Users/mac/Code/Joshua's Project"
```

Install the required libraries (only needed the first time):

```
pip3 install -r requirements.txt
```

Start the dashboard:

```
streamlit run dashboard/app.py
```

Your web browser opens automatically at <http://localhost:8501>. To stop it, click back on Terminal and press **Ctrl + C**. That's it — the trained models are already saved, so the dashboard works straight away.

7 How to put it online (free)

To get a public link you can share, use **Streamlit Community Cloud**. You will need a free **GitHub** account. In short:

- Upload the project to a GitHub repository (a `git init → git add → git commit → git push` sequence).
- Go to share.streamlit.io, sign in with GitHub, click **Create app**.
- Choose your repository and set the main file to `dashboard/app.py`, then **Deploy**.

After a few minutes you get a public address like `https://your-app.streamlit.app` . Full step-by-step instructions are in the file `dashboard/DEPLOY.md` .

8 What's in the project folder

File / folder	What it is
<code>crop_yield_dataset.csv</code>	The raw data
<code>maize_yield_data_preparation.ipynb</code>	Notebook: cleaning & preparing the data (stage ②)
<code>maize_yield_modeling.ipynb</code>	Notebook: training & comparing the models (stage ③)
<code>dashboard/app.py</code>	The dashboard web app (stage ④)
<code>outputs/models/</code>	The saved trained models the dashboard uses
<code>outputs/figures/</code>	All the charts
<code>requirements.txt</code>	The list of libraries to install

★ Quick glossary

Term	In plain words
Yield	How much maize a plot produces, in tonnes per hectare.
Feature	One input the model uses (e.g. rainfall).
Training / test set	Data used to teach the model / data kept aside to grade it fairly.
RMSE & MAE	Average size of the prediction error — smaller is better.
R² (R-squared)	Share of the variation in yield the model explains, from 0 to 1 — closer to 1 is better. Ours is ≈ 0.98 .
Accuracy	Percentage of predictions that land within 10% of the true yield.